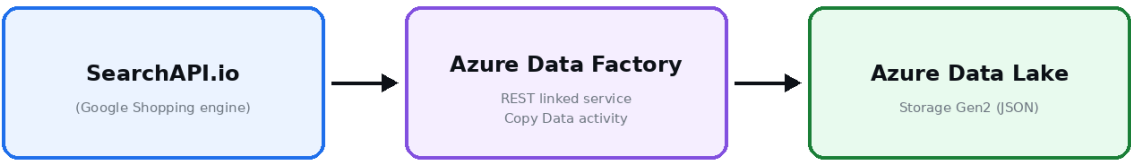


Fetching Google Shopping Results with an API, Landed in Azure

A real, filtered Google Shopping search — price range, location, sale status — pulled through a REST API and automatically saved to Azure Data Lake Storage using an Azure Data Factory pipeline. No manual downloads, no copy-pasting.



Pipeline flow: SearchAPI.io → Azure Data Factory → Azure Data Lake Storage

Source	SearchAPI.io — Google Shopping engine
Destination	Azure Data Lake Storage Gen2 (JSON)
Orchestration	Azure Data Factory — Copy Data activity
Query filters	product, price ceiling, location, sale status, images
Runtime	~22 seconds per run, ~84 KB written

OVERVIEW

What this project does

This is a follow-up to an earlier pipeline that pulled live Google News through SearchAPI.io into Azure. Here the same pattern is applied to **Google Shopping** instead — searching for a product (a PS5) with real filters like price range, location, and sale status, then landing the results as JSON in Azure Data Lake Storage via Azure Data Factory.

The build is split into four parts:

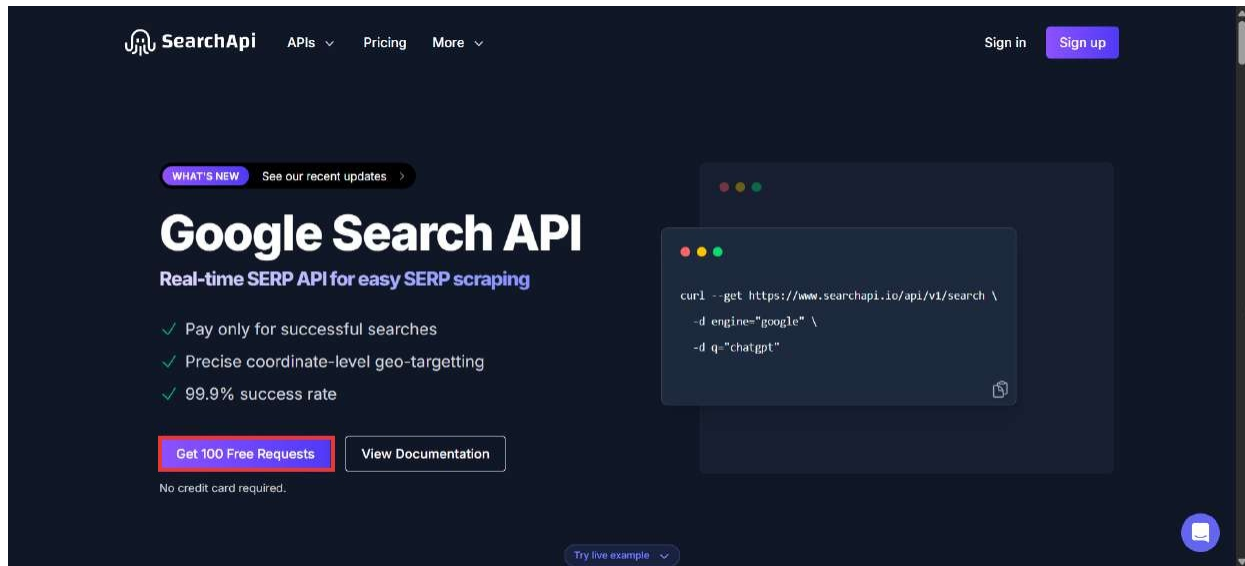
Part 1	Build a Google Shopping API URL with real search filters
Part 2	Set up Azure storage to hold the results
Part 3	Build the Data Factory pipeline connecting the two
Part 4	Run it, and verify the shopping results land in storage

PART 1

Building the Google Shopping API URL

SearchAPI.io turns a Google search into a simple API call. This time the target is Google Shopping, and the query needs real filters — not just a search term.

■ Start from SearchAPI's Google Search API



Signing up gives 100 free searches and an API key. From here it's a matter of picking the right engine and parameters for Google Shopping.

■ Assemble the URL with real shopping filters

LIVE API

```
API Key : APIKEY
Google Shopping API Key : https://www.searchapi.io/api/v1/search?engine=google_shopping&q=PS5&gl=in&price_max=85000Rs&location=Kerala_India&is_on_sale=true&include_base_images=true&api_k APIKEY
Base URL: https://www.searchapi.io
Endpoint : /api/v1/search
Engine: Google -- search engine
q = PS5 -- search query
gl = in --location India
Api key -- our api key
```

BEFORE

Just knowing the engine name (google_shopping) and a search term.

AFTER

A complete URL with several filters chained together — see the request below. That's: search for "PS5", in India (gl=in), priced under Rs 85,000, near Kerala specifically, only items on sale, and include product images in the response.

```
GET /api/v1/search
?engine=google_shopping
&q=PS5
&gl=in
&price_max=85000Rs
&location=Kerala_India
&is_on_sale=true
&include_base_images=true
&api_key=<your key>
```

TIP

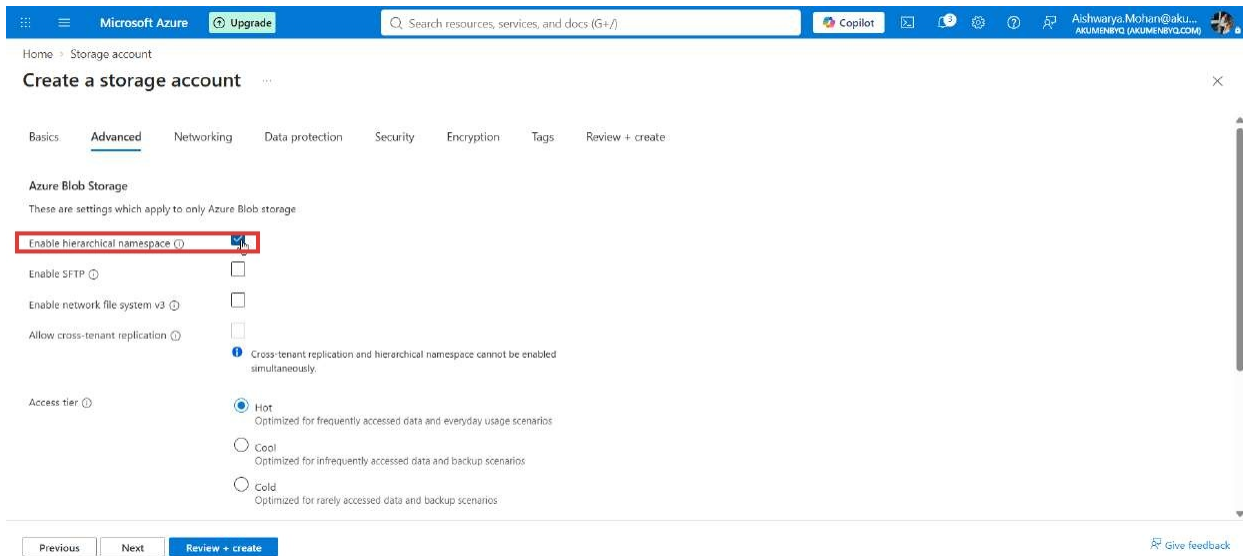
Key blurred in the screenshot above — this project lives on a public GitHub repo, never commit a real API key.

PART 2

Setting Up Storage in Azure

A storage account with hierarchical namespace turned on, so it behaves as a proper Data Lake, plus a container to hold the output.

■ Enable hierarchical namespace



Microsoft Azure Upgrade Search resources, services, and docs (G+/I) Copilot Aishwarya.Mohan@aku... AKUMENBYQ (AKUMENBYQ.COM)

Home > Storage account

Create a storage account

Basics **Advanced** Networking Data protection Security Encryption Tags Review + create

Azure Blob Storage
These are settings which apply to only Azure Blob storage

Enable hierarchical namespace ☒ 

Enable SFTP ☐

Enable network file system v3 ☐

Allow cross-tenant replication ☐
Cross-tenant replication and hierarchical namespace cannot be enabled simultaneously.

Access tier 
☒ **Hot**
Optimized for frequently accessed data and everyday usage scenarios

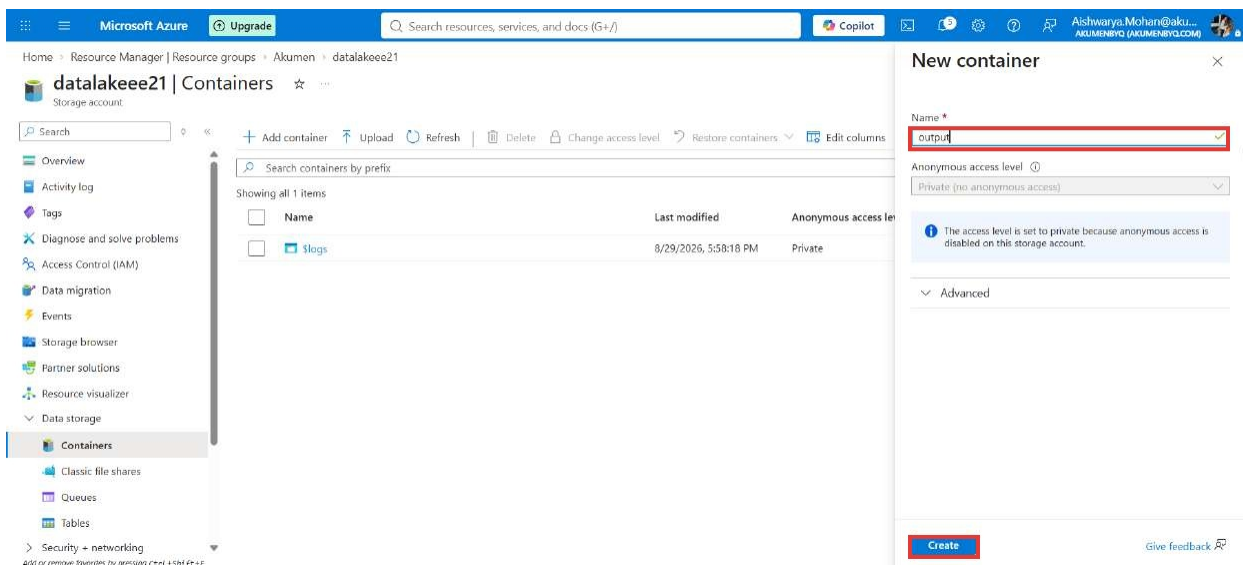
☐ **Cool**
Optimized for infrequently accessed data and backup scenarios

☐ **Cold**
Optimized for rarely accessed data and backup scenarios

Previous Next **Review + create** Give feedback

This checkbox under Advanced is what makes the storage account behave as **Azure Data Lake Storage Gen2** instead of plain Blob Storage — needed for the folder-style organization used later.

■ Create the “output” container



Microsoft Azure Upgrade Search resources, services, and docs (G+/I) Copilot Aishwarya.Mohan@aku... AKUMENBYQ (AKUMENBYQ.COM)

Home > Resource Manager > Resource groups > Akumen > datalakeee21

datalakeee21 | Containers

Storage account

Search Add container Upload Refresh Delete Change access level Restore containers Edit columns

Showing all 1 Items

Name	Last modified	Anonymous access level
logs	8/29/2026, 5:58:18 PM	Private

New container

Name * **output** 

Anonymous access level 
Private (no anonymous access)

The access level is set to private because anonymous access is disabled on this storage account.

Advanced

Create Give feedback

A container named output, kept Private since there's no reason for this data to be public.

■ Add Data Factory to the resource group

The screenshot shows the Microsoft Azure portal interface for the 'Akumen' resource group. The left sidebar contains navigation links for Overview, Activity log, Access control (IAM), Tags, Resource visualizer, Events, Settings, Cost Management, Monitoring, Automation, and Help. The main content area is titled 'Overview' and includes a search bar, a 'Create' button, and a 'Manage view' dropdown. Below this, the 'Resources' tab is selected, showing a table of resources. The table has columns for Name, Type, and Location. Two resources are listed: 'datafactoryyyyy21' (Data factory (V2)) and 'datalakeee21' (Storage account), both located in 'Central India'. The bottom of the page shows a status bar with 'Showing 1 - 2 of 2. Display count: auto' and a 'Give feedback' link.

Name	Type	Location
datafactoryyyyy21	Data factory (V2)	Central India
datalakeee21	Storage account	Central India

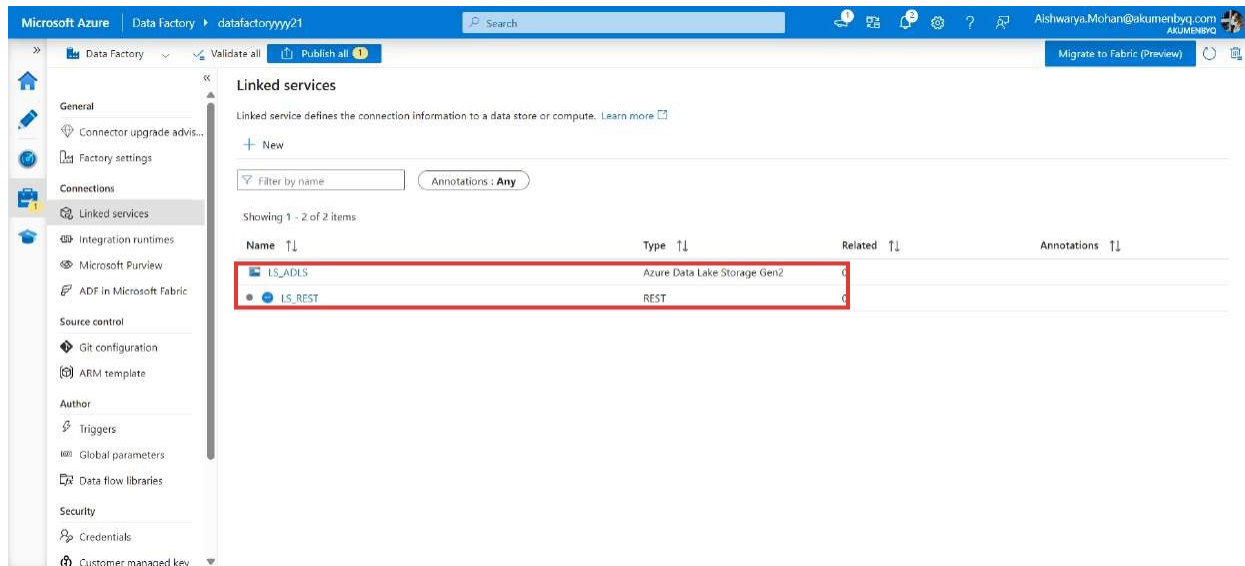
Both the storage account (data lakeee21) and the new Data Factory (datafactoryyyyy21) sit in the same **Akumen** resource group.

PART 3

Wiring Up Data Factory

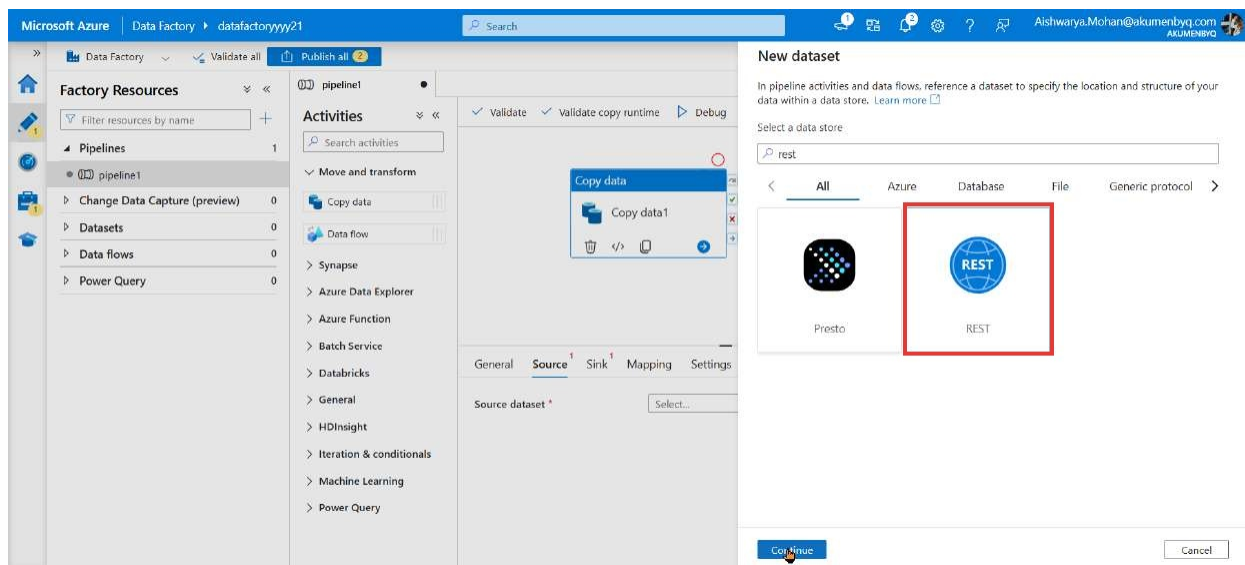
Since this reuses the pattern from the News pipeline, the linked services step is quick — same two connector types, pointed at this Data Factory instance.

Set up both linked services



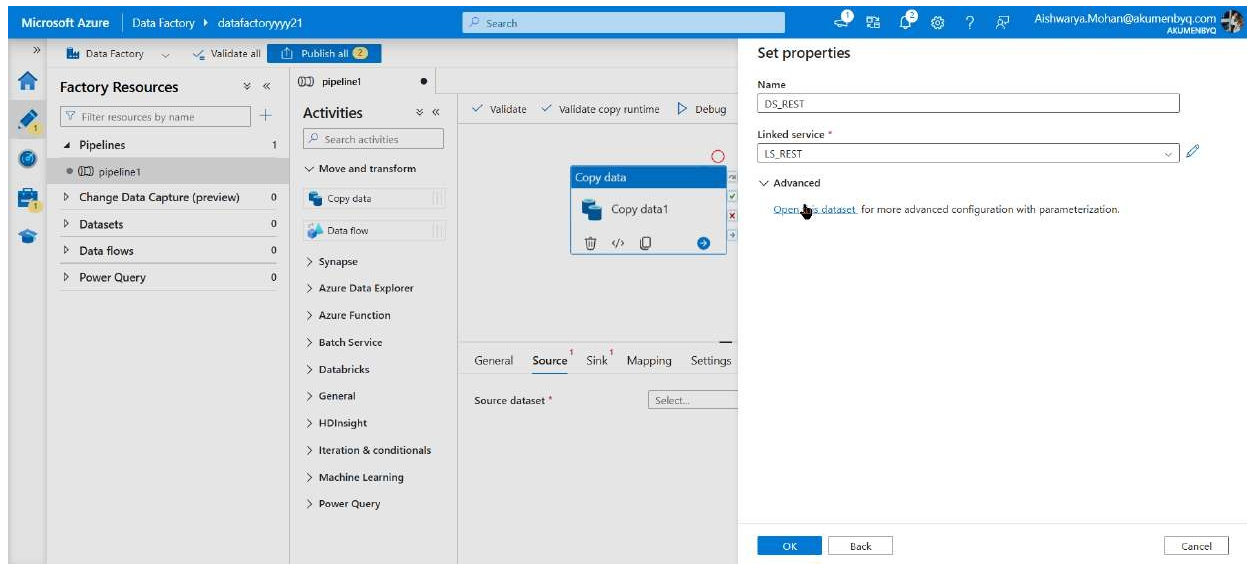
A **REST** linked service for SearchAPI, and an **Azure Data Lake Storage Gen2** linked service for the storage account — LS_REST and LS_ADLS.

Create the source dataset



Picked the REST connector for the source — this dataset represents the SearchAPI Shopping request.

■ Name it and link it



Named it DS_REST, linked to LS_REST. The full request URL — engine, query, filters, and key — is set on this dataset.

- Preview the live results

The screenshot shows the Microsoft Azure Data Factory interface. The top navigation bar includes 'Microsoft Azure', 'Data Factory', and the data lake name 'datafactoryyyyy21'. The left sidebar, titled 'Factory Resources', lists various components: Pipelines (1), Change Data Capture (0), Datasets (1), Data flows (0), and Power Query (0). The 'Datasets' section is expanded, showing a dataset named 'DS_REST'. The main window is titled 'Preview data' and displays a JSON object for the 'search' dataset. The JSON object contains search metadata and parameters. The right sidebar shows the 'Properties' tab for the dataset, with fields for 'Name' (pipeline1) and 'Description'. A 'Cancel' button is visible at the bottom right.

Factory Resources

- Pipelines 1
- Change Data Capture (preview) 0
- Datasets 1
- Data flows 0
- Power Query 0

Preview data

Linked service: LS_REST

Object:

```
{
  "search_metadata": {
    "id": "search_66ZMRV4jp7t1lqoRak2X071E",
    "status": "Success",
    "created_at": "2026-08-29T12:57:20Z",
    "request_time_taken": 8.55,
    "parsing_time_taken": 0.04,
    "total_time_taken": 8.59,
    "request_url": "https://www.google.com/search?q=P55&under=85000Rs&og=P55&under=85000Rs&gl=in&hl=en",
    "html_url": "https://www.searchapi.io/api/v1/searches/search_66ZMRV4jp7t1lqoRak2X071E.html",
    "json_url": "https://www.searchapi.io/api/v1/searches/search_66ZMRV4jp7t1lqoRak2X071E"
  },
  "search_parameters": {
    "engine": "google_shopping",
    "q": "P55",
    "location": "Kerala_India",
    "location_used": "Kerala,India",
    "hl": "en",
    "gl": "in",
    "is_on_sale": "true",
    "price_max": "85000Rs",
    "include_base_images": "true"
  },
  "search_information": {
    "query_displayed": "P55 under 85000Rs",
  }
}
```

Properties

General Related

Name *

pipeline1

Description

Annotations

+ New

Cancel

The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists 'Pipelines' and 'pipeline1'. The main 'Preview data' pane shows a JSON snippet of the pipeline's options, which includes game categories like 'PlayStation Games', 'Video Game Consoles', and 'Video Games'. The right sidebar shows the 'Properties' pane with 'General' selected, displaying the pipeline's name and description.

Microsoft Azure | Data Factory | datafactoryyyyy21

Search

Migrate to Fabric (Preview)

Factory Resources

Filter resources by name

- Pipelines 1
 - pipeline1
- Change Data Capture (preview) 0
- Datasets 1
 - DS_REST
- Data flows 0
- Power Query 0

Preview data

```
{
  "position": 2,
  "product_id": "445450090790726337",
  "title": "Sony PlayStation 5 Video Game Console",
  "product_link": "https://www.google.com/search?ibp=oshop&q=PSS&prds=catalogid:445450090790726337,i",
  "offers_link": "https://www.google.com/search?ibp=oshop&q=PSS&prds=catalogid:445450090790726337,g",
  "price": "\u00a34,999.12",
  "extracted_price": 43999.12,
  "original_price": "\u00a35,999",
  "extracted_original_price": 49999.0,
  "rating": 3.0,
  "reviews": 176,
  "tag": "12% OFF",
  "delivery": "Free delivery",
  "delivery_return": "7-day returns",
  "seller": "vlebazaraa",
  "thumbnail": "data:image/webp;base64,Uk1GRk43AABXRUXQVIA4E1jAADQpwCpAwS6A6ZlBP0c3jEUlaEQvegiAPE",
  "product_token": "eyJ3IjoifHwPM1HVuZGVyIDg1MDAwUmMlC0Rhc1E6Im1uIiwiaW50IjI1b1E1InV1bGU1O13K0NB3VF",
},
{
  "position": 3,
  "product_id": "17481386995326658839",
  "title": "Sony PlayStation 5 Digital Edition",
  "product_link": "https://www.google.com/search?ibp=oshop&q=PSS&prds=catalogid:17481386995326658839",
  "offers_link": "https://www.google.com/search?ibp=oshop&q=PSS&prds=catalogid:17481386995326658839",
  "price": "\u00a34,350",
  "extracted_price": 41356,
  "original_price": "Usually \u00a34,720",
  "extracted_original_price": 47229.0,
  "rating": 4.5,
  "reviews": 6600,
}
```

Properties

General Related

Name *

pipeline1

Description

Annotations

+ New

Cancel

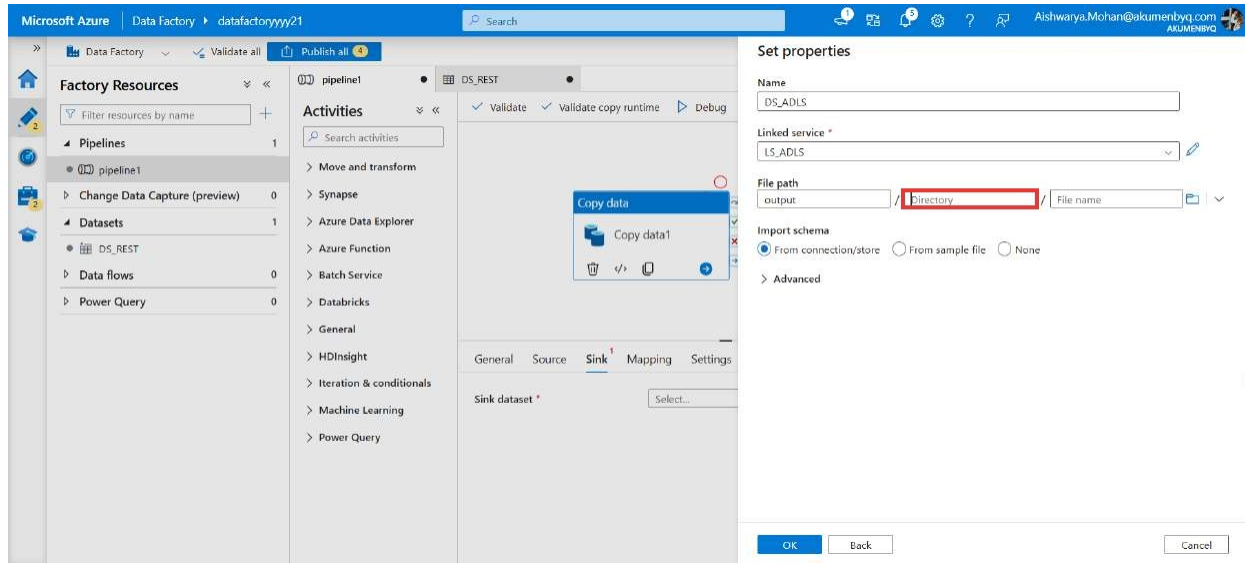
The preview confirms every filter took effect — `search_parameters` shows `q: "PS5"`, `gl: "in"`, `price_max: "85000Rs"`, `is_on_sale: "true"` — all present and correct.

Scrolling further shows real product listings: a Sony PlayStation 5 console at Rs 43,999 (12% off), a Digital Edition at Rs 41,356 — with ratings, review counts, delivery info, and seller names. This is live shopping data, not a mock response.

PART 3

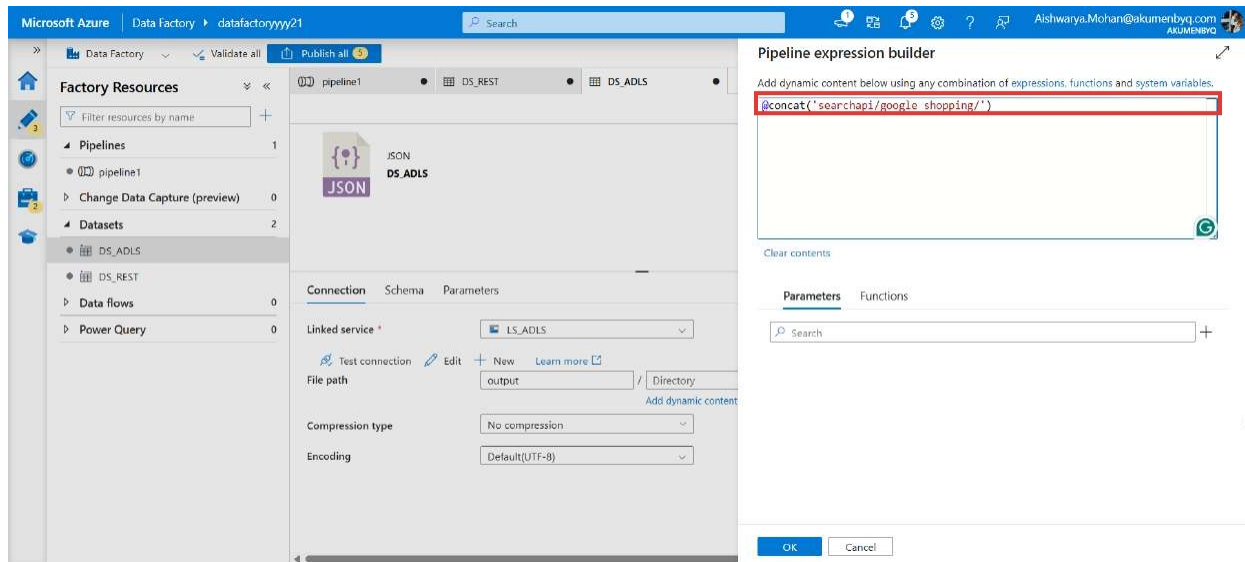
Saving the Output

Create the sink dataset



Named DS_ADLS, linked to LS_ADLS, sitting inside the output container.

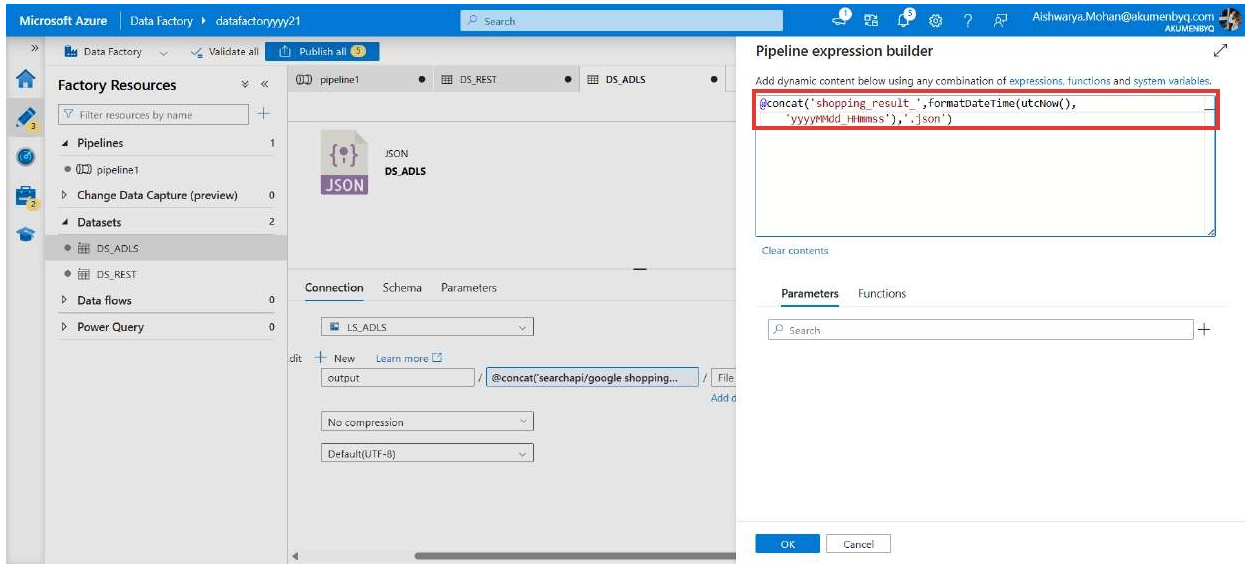
Set a fixed destination folder



```
@concat('searchapi/google shopping/')
```

A simple fixed path this time — no date-based subfolders needed, since this is a one-off product search rather than a recurring news feed.

Build the filename — no gotchas this time

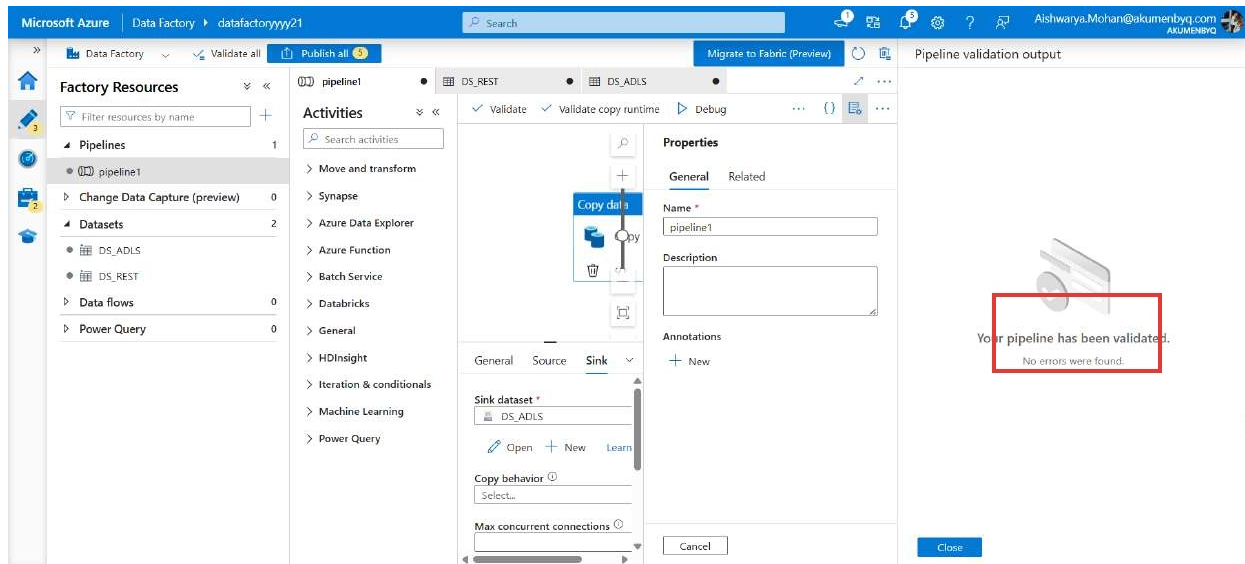


```
@concat(  
  'shopping_result_',  
  formatDateTime(utcnow(), 'yyyyMMdd_HHmss'),  
  '.json'  
)
```

TIP

Learned from the News project: date-format tokens are case-sensitive, and colons aren't allowed in file paths. This filename went right on the first try — uppercase MM for month, dashes and underscores instead of colons.

■ Validate the pipeline



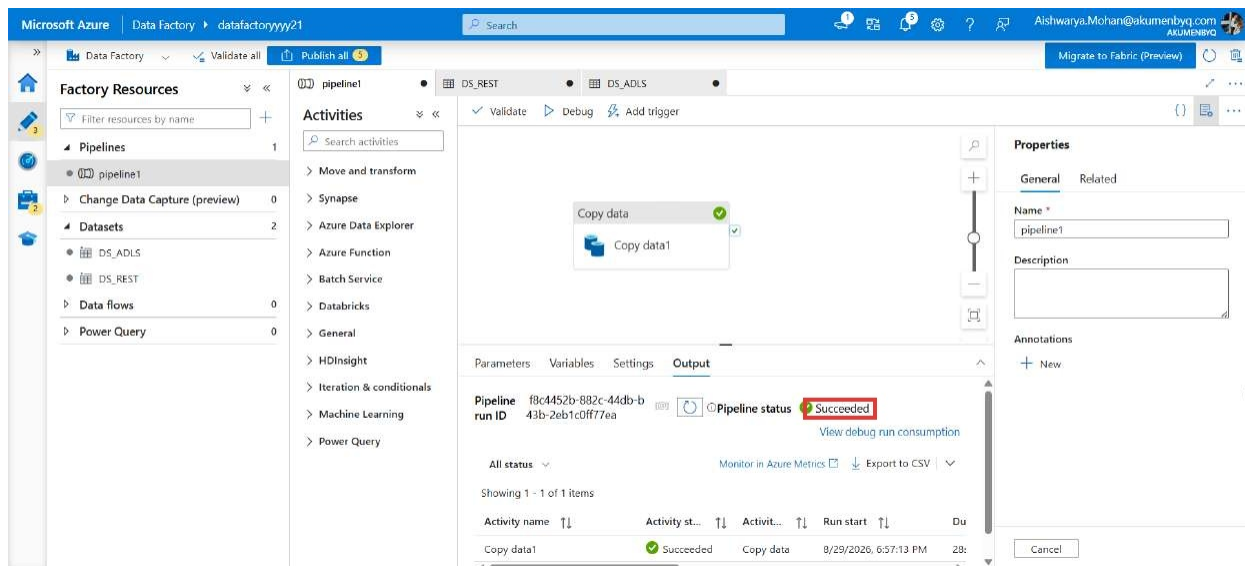
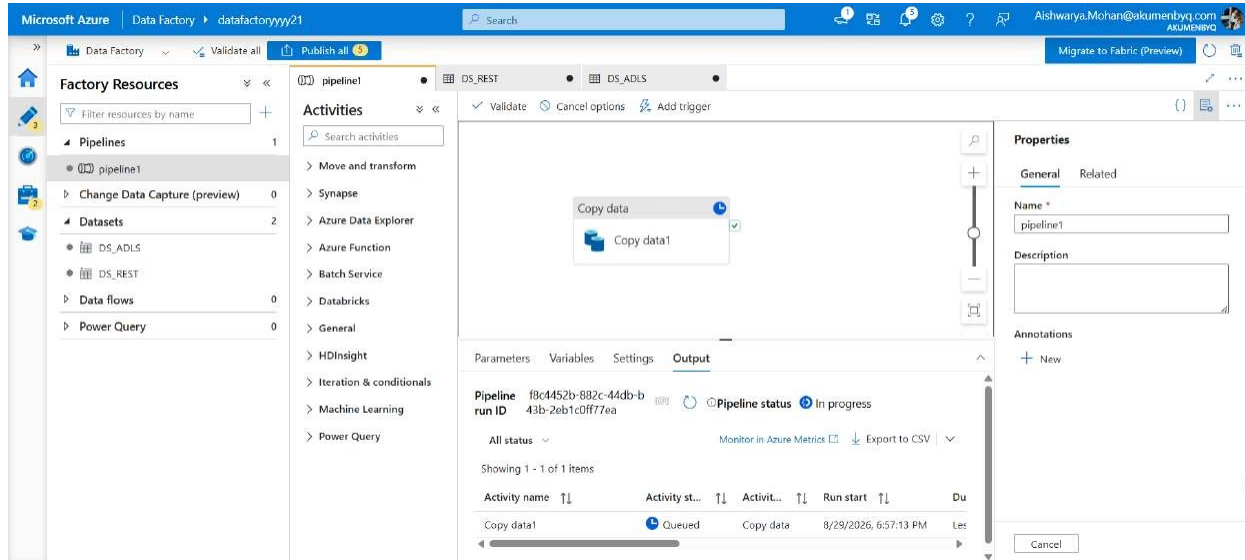
Validate checks the whole pipeline for configuration errors before running for real. “No errors were found” confirms the source, sink, and expressions are wired up correctly.

PART 4

Running It and Checking the Result

Everything validated — time to run the pipeline and confirm the shopping results land in storage.

Run it



BEFORE

The run sits at Queued / In progress right after starting.

AFTER

A green **Succeeded** status once the Copy Data activity finishes.

■ Check the copy statistics

The screenshot shows the 'Details' view of a copy activity in Microsoft Azure Data Factory. The activity is a REST activity that has succeeded, copying data to Azure Data Lake Storage Gen2 (Region: Central India). The statistics are as follows:

Category	Value
Data read	165.42 KB
Objects read	1
Peak connections	1
Data written	83.829 KB
Files written	1
Rows written	1
Peak connections	1

Copy duration: 00:00:22
Throughput: 82.71 KB/s

Start time: 8/29/2026, 6:57:14 PM
Used DUs: 4
Used parallel copies: 1
Duration: 00:00:22

Details	Working duration	Total duration
Queue	00:00:08	00:00:10
Transfer	00:00:07	00:00:10

About 165 KB read from the API, roughly 84 KB written to storage, 1 file, 1 row, done in 22 seconds — heavier than the News pipeline, which makes sense given all the product detail (images, prices, ratings) in a shopping result.

■ Find the file in storage

The screenshot shows the 'output' container in Microsoft Azure Storage Explorer. The file 'shopping_result_20260829_132714.json' is highlighted in the list of items.

Name	Last modified	Access tier	Blob type	Size	Lease state
shopping_result_20260829_132714.json	8/29/2026, 6:57:37 PM	Hot (Inferred)	Block blob	81.86 KiB	Available

Following output > searchapi > google shopping leads to the auto-named file: shopping_result_20260829_132714.json.

Peek inside the file

The screenshot shows the Microsoft Azure portal interface. On the left, the 'output' container is selected, showing a list of blobs. The main pane displays the content of the blob 'searchapi/google shopping/shopping_result_20260829_132714.json'. The blob is a JSON file containing search metadata and results. The JSON structure includes fields like 'search_metadata', 'request_time_taken', 'parsing_time_taken', 'total_time_taken', 'request_url', 'html_url', 'json_url', 'search_parameters', 'search_information', and 'filters'. The search parameters include 'engine': 'google_shopping', 'q': 'PSS', 'location': 'Kerala_india', 'location_used': 'Kerala, India', 'hl': 'en', 'is_on_sale': 'true', 'price_max': '85000Rs', 'include_base_images': 'true', and 'search_information' includes 'query_displayed': 'PSS under 85000Rs' and 'detected_location': 'Kerala'. The filters section contains 'options' with 'text': 'On sale'.

The screenshot shows the Azure portal interface with the JSON content of the blob displayed in a viewer. The JSON structure is expanded, showing the following details:

- search_metadata**:
 - engine: "google_shopping"
 - q: "PSS"
 - location: "Kerala_india"
 - location_used: "Kerala, India"
 - hl: "en"
 - is_on_sale: "true"
 - price_max: "85000Rs"
 - include_base_images: "true"
- search_information**:
 - query_displayed: "PSS under 85000Rs"
 - detected_location: "Kerala"
- filters**:
 - options: [{"text": "On sale"}]

The right pane shows a table of the JSON content, listing fields like 'created_at', 'html_url', 'id', 'json_url', 'parsing_time_taken', 'request_time_taken', 'request_url', 'status', and 'total_time_taken'.

Opening the blob confirms a full, valid JSON response — search metadata, every parameter used, and the shopping results themselves. Ready for anything downstream: a price-tracking dashboard, a Databricks notebook, or a scheduled trigger to re-run this search daily and watch for price drops.

KEY TAKEAWAYS

- ✓ Reused an existing Azure connector pattern instead of rebuilding from scratch
- ✓ Applied real query filters (price, location, sale status) through URL parameters
- ✓ Avoided both date-formatting gotchas from the previous build on the first try
- ✓ End-to-end run: ~22 seconds, fully automated, zero manual steps

One filtered API URL, a reused Azure pipeline pattern, and a live Google Shopping snapshot saved automatically — ready to run on a schedule.